

Toward Agentic Coding: Designing for Habit Formation in AI-Assisted Data Work

Noel Konagai
Databricks
Seattle, Washington, USA
noel.konagai@gmail.com

Margaret T Lynn
Databricks
Seattle, Washington, USA
maggie.lynn@databricks.com

Eva Snee
Databricks
Seattle, Washington, USA
eva.snee@gmail.com

Abstract

Advances in AI are rapidly adding new features to code authoring tools that can write entire notebooks from natural language prompts. Despite this, most data and AI professionals continue to use manual programming for everyday data analytics tasks. Building on BJ Fogg’s behavior model and habit formation literature, we explore this adoption gap: if manual coding is an established habit, we argue agentic coding experiences ought to be designed with habit formation in mind. We propose four preliminary design principles to support a shift from reactive to proactive agentic engagement: (1) leverage organic disruptions during users’ workflows, (2) design first experiences for plausible wins, (3) reduce friction with contextual prompting affordances, and (4) offer follow-up affordances.

CCS Concepts

• Human-centered computing → Natural language interfaces; HCI theory, concepts and models; User studies.

Keywords

Habit formation, agentic coding, behavioral design, AI adoption, code authoring, interaction design

ACM Reference Format:

Noel Konagai, Margaret T Lynn, and Eva Snee. 2026. Toward Agentic Coding: Designing for Habit Formation in AI-Assisted Data Work. In *Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems (CHI EA '26)*, April 13–17, 2026, Barcelona, Spain. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3772363.3799043>

1 Introduction

Most data and AI professionals continue to use manual programming for their daily data analytics tasks. From university training to documentation and peer norms, multiple artifacts reinforce the idea that coding is done by hand. At the same time, AI is rapidly changing what code authoring means to those in industry; data practitioners now have tools ranging from linters and autocompletes to copilots that can write entire data science notebooks from natural language prompts.

In an internal data practitioner study at Databricks we observed that even enthusiastic users of the data science agentic programmer often struggle to get started with letting an agent code on their

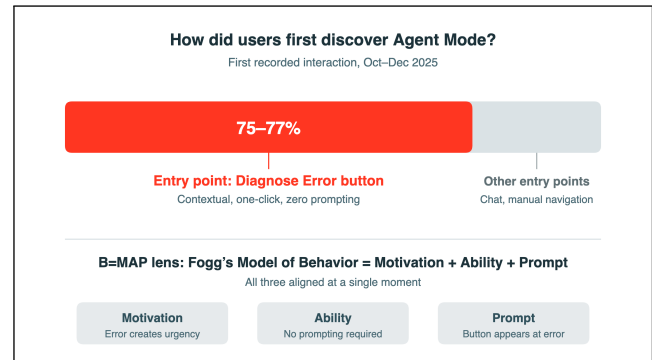


Figure 1: Discovery paths into Agent Mode: 75–77% of users who first interacted with Agent Mode (Oct–Dec 2025) discovered it through the Diagnose Error button, interpreted through Fogg’s B=MAP framework.

behalf. Many return to manual workflows after one-off successes, suggesting that agent use has not yet consolidated into a stable habit. This gap motivates our turn toward a habit formation perspective: if manual coding is an established habit, agentic coding experiences ought to be designed with habit building in mind.

This work builds on previous research on habit formation and behavior change (e.g., Lally et al. [8]), the time course of habit formation (Fogg’s B=MAP behavior model [5]), context change (Verplanken et al. [16]), and empirical studies of AI coding assistants (e.g., Barke et al.’s GitHub Copilot study [1]). We propose a set of design principles for agentic coding experiences that leverage habit formation mechanisms, grounded in the above literature and internal UX research on data science programming agent usage at Databricks. The design principles are intended to increase the sustained voluntary adoption of agentic coding tools by: aligning interventions with our users’ existing goals and workflows, reducing cognitive overhead, optimizing positive first experiences, and supporting the shift towards agentic coding behavior.

2 Related Work

In this section, we cover related work in the areas of (1) programming habits and (2) habit replacement.

2.1 Code Authoring as an Entrenched Habit

Wood & Neal [17] describe habits as behaviors that, through repetition in stable contexts, become automatically cued by those contexts and require little conscious attention to execute. While such habits are functional because they free up cognitive resources, they are



This work is licensed under a Creative Commons Attribution 4.0 International License. *CHI EA '26, Barcelona, Spain*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2281-3/26/04

<https://doi.org/10.1145/3772363.3799043>

also relatively inflexible once established. Authoring code can be understood as a behavior surrounded by habits; reaching for a particular IDE, opening the same documentation website, or beginning every file with a familiar block of import statements are all entrenched routines that structure how code authors start and sustain their work. Caliskan-Islam et al. [3] demonstrate that an individual's coding style is consistent enough to identify programmers from source code with over 94% accuracy, and these stylistic fingerprints persist even after compilation into executable binaries [2][13]. Liang et al. found that despite being motivated by efficiency gains, developers do not accept AI programming assistants' suggestions at a high frequency [9]. Russo found that compatibility with existing workflows, rather than perceived usefulness, was the strongest predictor of AI tool adoption [15]. Together, these findings suggest that existing coding workflows act as a strong default that AI tools must integrate into, starting with repetitive or friction-heavy tasks. If coding-related habits are both deeply entrenched and somewhat inflexible, a key design question becomes: How might we deliberately cultivate new habits for our users to reach for agentic coding experiences as the default starting point for code authoring?

2.2 Replacing, Not Breaking, Coding Habits

It might be tempting, from a design point of view, to suppress manual coding by degrading the affordances that support it. If manual coding becomes harder, users with the same goals might naturally turn to alternative means. However, research on habit change suggests that simply trying to break a habit is a suboptimal strategy. Durable change typically requires replacing the old response with a new one. Habit reversal work, synthesized by Miltenberger, Fuqua, and Woods [11], shows that effective interventions do not merely instruct people to stop a behavior. Instead, they train an alternative action to perform when the old cue or urge appears, and contingent practice of that alternative is what drives lasting reductions in the original habit. Although this research originates in clinical contexts, we propose the core mechanism (training a competing response to the same cue) is applicable to coding behaviors, which share the same cue-dependent, repetitive structure. The habit discontinuity hypothesis [16] further suggests that when contextual cues are disrupted, a window opens in which habitual behavior is more deliberately reconsidered, creating an opportunity for new behaviors to take root. Although existing estimates of time to habit formation are derived from health behaviors, they consistently suggest that habit formation is a multi-month process [6] rather than something that stabilizes after a handful of repetitions. As a corollary, we expect agentic coding habits to form over similar time scales. Taken together, this literature suggests that agentic coding is more likely to take hold when introduced as a competing response within familiar coding contexts, particularly during moments of disruption, rather than by degrading manual coding affordances.

3 B=MAP Framework for Agentic Coding Habit Formation

Fogg's Behavior Model [5] proposes three components for a behavior to occur: motivation, ability or ease of performing a certain

behavior, and a timely prompt. All three need to happen simultaneously to trigger a behavior. While the Technology Acceptance Model [4] explains adoption in terms of individuals' perceived usefulness and ease of use, and Diffusion of Innovation theory [12] describes how innovations spread through social systems over time, neither framework addresses how to design interventions that trigger a specific behavior at a specific moment. B=MAP fills this gap by specifying the conditions that must converge for a behavior to occur, making it well suited to our goal of designing contextual interventions that replace entrenched coding habits. Since coding habits are repeated behaviors, we can understand coding-related behaviors with Fogg's Behavior Model described below.

Motivation. Fogg notes down three sub-types of motivation: pleasure and pain, hope and fear, social acceptance and rejection. In coding contexts, agentic experiences largely fall under the hope and fear dimension. If a user believes that agents can erroneously delete tables and databases, they will not be motivated to use the agentic experience. Conversely, with the hope that an agent can perform repetitive or tedious tasks faster than a human, the user is motivated to use an agent.

Ability (elements of simplicity). Fogg identifies six elements of simplicity that reduce friction in performing a desired behavior: time, money, physical effort, 'brain cycles,' social deviance, and non-routine. In the context of agentic coding, three elements are particularly relevant: time, brain cycles, and non-routine behavior. A user may want to avoid spending time on repetitive tasks and naturally turn to agentic coding experiences for acceleration. However, prompting an agent presents a significant friction point: it is a new skill that demands cognitive resources (brain cycles), requires time investment to learn, and represents a non-routine task for many. Roy et al. [14] supports this; the authors argue that prompt engineering should form part of advanced CS curricula, acknowledging that effective prompting requires explicit training and iterative practice (see also [10]). This creates a design challenge: if using agents requires learning prompt engineering, the friction of acquiring this new skill may outweigh the benefits of automation, especially for first-time users. This suggests that agentic experiences should minimize the cognitive load, time, and non-routine aspects of prompting, potentially by eliminating the need for manual prompt composition entirely through contextual affordances and pre-filled suggestions.

Prompt. Not to be confused with prompting agents, in Fogg's behavior model [5] prompts are also known as cues or calls to action. He identifies three types of triggers: spark, facilitator, and signal. A spark boosts motivation by surfacing hope of a positive outcome, while a facilitator reduces friction for users who are already motivated to act. For the reason that prompting agents is a new skill, designing facilitator-type prompts lowers the barrier to invoking agentic assistance and supports habit development, e.g. a button with a pre-filled prompt to be sent to the agent.

3.1 Two Paths to Agentic Adoption: Acceleration and Exploration

While Fogg's model provides a general framework for understanding behavioral change, understanding what specifically motivates

developers to use AI coding assistance requires examining the concrete goals they pursue during coding work. Barke et al. [1] argued that AI coding co-pilots such as Github Copilot largely mirror the dual-process of cognition popularized by Kahneman [7]. The first mode is acceleration, when users want to accelerate their current work and get to their goals faster using AI. The second mode is exploration, when users are unsure how to proceed and need to deliberately slow down to process the information and problem-solve. We return to this distinction when proposing design principles.

4 Diagnose Error: A Case Study in B=MAP-Driven Adoption

We present a case study from Databricks that demonstrates Fogg’s behavioral model in practice and resulted in high initial agent adoption. Databricks launched an agentic data science programming experience that we will refer to as Agent Mode for the rest of this paper. Agent Mode was launched in Beta on September 3, 2025. The telemetry reported here covers October through December 2025, the first three months of broad beta availability. The data reflect all the users whose first recorded Agent Mode interaction occurred during this period, the vast majority of whom (approximately 99%) were users external to Databricks. No filtering by role or expertise was applied; users included data practitioners with varying roles and levels of programming experience. We note that telemetry logging began approximately four weeks after the Beta launch; a small number of early beta adopters whose first interaction preceded the logging start date may appear in the October cohort.

With the launch of Agent Mode, we introduced a one-click ‘Diagnose Error’ button, which appears when a notebook cell results in an error that the user needs to be aware of and fix. A typical workflow would be to open a side panel to find the data science programming agent and prompt it to fix the error. Instead of manual navigation and prompting, this one-click button automatically sends the relevant context needed for the agent to start building bug fixes. During the first three months of beta availability (October through December 2025), our internal telemetry consistently showed that approximately 75–77% of users whose first recorded Agent Mode interaction occurred in each month discovered it through the one-click Diagnose Error button rather than through manual navigation or other entry points. The remaining users primarily discovered Agent Mode through the general-purpose assistant chat available across the Databricks platform or by manually navigating to the agent and typing a prompt there.

4.1 Mapping the Agentic Diagnose Error Workflow to B=MAP

Motivation: We introduce the Diagnose Error button at a moment when users are already motivated to resolve the problem (their immediate goal is to fix the error and resume coding). By aligning with users at the moment of frustration, we are able to channel their motivation into an agentic solution that promises faster resolution. Additionally, by reducing the time spent troubleshooting, we help users regain coding flow faster, conserving cognitive resources.

Ability: We made it easier to use the agent by reducing the time investment, the mental effort needed to gain value, and the friction of performing a non-routine task. The button automatically

provides the context and information needed to solve a bug without extra effort.

Prompt: Critically, this design detail also eliminates the need to manually prompt the agent (a significant barrier for users unfamiliar with AI prompting). This ‘facilitator trigger’ reduces friction for users who are already motivated to pursue the agentic experience. It also contains a ‘spark trigger’; by surfacing hope that the agent can fix the error, it boosts motivation for users who might otherwise hesitate to try Agent Mode.

4.2 B=MAP in Practice: Understanding the High Adoption of the Diagnose Error Button

As all three elements of the behavior model happen concurrently, we observe that approximately 75–77% of users whose first recorded Agent Mode interaction took place during each month of the beta period (October–December 2025) discovered it through this button. Although this metric reflects initial discovery rather than sustained habitual use, it suggests the design was effectively aligned with B=MAP’s three elements. We leveraged the button as an affordance to facilitate users getting closer to their end-goal of resolving the bug. As bugs recur throughout the coding process, these repeated encounters may provide sufficient repetition to support habit formation around the Diagnose Error button and thus Agent Mode.

4.3 From Single Use to Habit: What’s Still Needed

A single interaction with Diagnose Error does not constitute a habit. For durable agentic coding habits to form, this behavior must repeat consistently in stable contexts possibly over months. Moreover, error diagnosis is inherently reactive as it is triggered only by bugs. To build habits where users proactively reach for agents during both acceleration and exploration tasks, not just error recovery, we need additional design principles beyond contextual affordances.

5 Proposed Design Principles for Agentic Coding Habit Formation

To support a shift from reactive to proactive agentic engagement, we identified four design principles grounded in Fogg’s B=MAP framework and our case study observations. These principles are preliminary, derived from the Diagnose Error case study and the habit formation literature reviewed above; they represent proposed design directions rather than empirically validated guidelines. Future work will empirically investigate their effect on sustained agentic coding adoption.

5.1 Leverage Organic Disruptions

A key insight from the Diagnose Error case study is that organic disruptions are optimal moments for habit replacement. When users encounter errors, they experience high motivation and focused attention, which are conditions that create an ideal window to introduce agentic alternatives to manual coding. Rather than requiring users to proactively seek out agents, a contextually embedded agent button provides the path of least resistance at the moment of peak need. This principle is supported by the habit discontinuity hypothesis, which shows that disruptions to existing

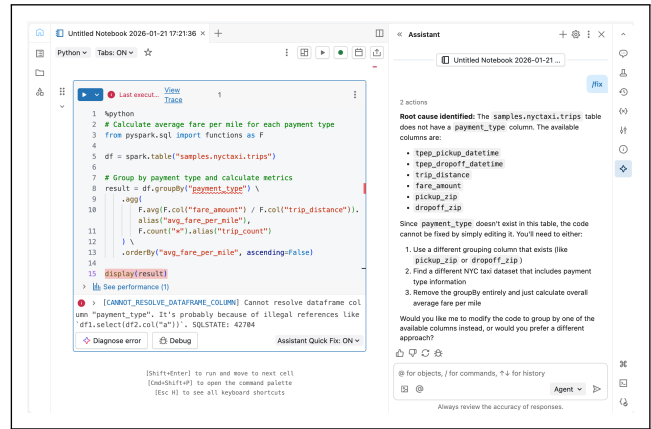
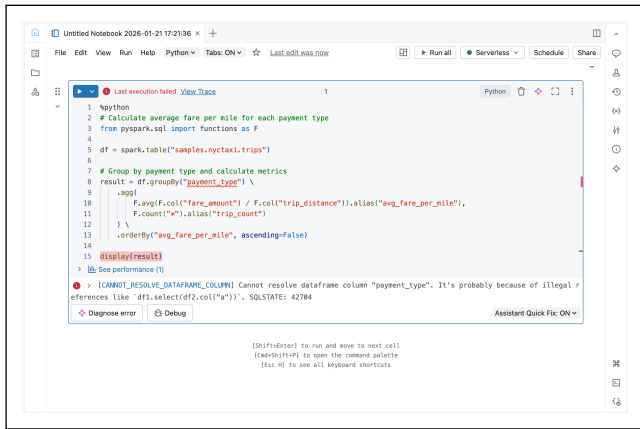


Figure 2: (Left) The error interrupts the user’s workflow, creating high motivation to fix it. Without this button the user needs to organically discover the entry point to the chat side panel with the agent. (Right) After clicking, the agent side-panel opens with pre-filled error context requiring zero prompting effort from the user, which aligns with ability and a facilitator prompt concepts.

routines create windows in which new behaviors can take root and compete with entrenched habits [16].

5.2 Design First Experiences for Plausible Wins

For first-time users, the concept of agentic autonomy can be unfamiliar, and users may have doubts about the efficacy of agentic coding [9]. Even users familiar with Q&A-style AI tools may hesitate to let an agent take actions on their behalf. By starting with low-stakes tasks likely to yield tangible value (such as time saved or cognitive effort reduced), early agentic interactions can create a foundation for habit formation. This principle reflects Barke et al.’s finding [1] that both acceleration and exploration can drive repeated use, provided the outcomes are visible and valuable.

5.3 Reduce Friction with Contextual Prompting Affordances

Prompting remains an unfamiliar skillset for many developers [10, 14]. Requiring users to learn prompt engineering creates unnecessary friction and cognitive load [9], a barrier that undermines adoption regardless of motivation or perceived value. Instead, contextual buttons or suggestions can initiate agent interactions with ease, eliminating the need for manual prompt composition.

Examples include: "Refactor this cell," "Explain this error," or "Generate documentation." Each button infers user intent from context and pre-fills the agent prompt. The Diagnose Error button exemplifies this approach; it required no prior prompting experience from users, yet the system successfully inferred intent from the error context. This design directly aligns with Fogg’s concept of ability in the B=MAP model as it removed the prompting barrier, ensuring that motivation and timely prompts translate into action.

5.4 Offer Follow-up Affordances

Habit formation requires not just single successes, but repeated engagement in the same context [8]. After a successful agent interaction, the habit-formation momentum can dissipate if users

must manually initiate the next step. Instead, providing contextually relevant next-step suggestions that capitalize on the user’s current focus and motivation can contribute to habit formation. For example, if the agent fixes a bug, the system could predict the most plausible next steps to complete the analysis in the notebook environment without requiring users to initiate new actions from scratch. By predicting plausible next actions based on the notebook environment’s state, this reduces friction and keeps users engaged in the agentic workflow, maintaining the conditions necessary for habit formation.

6 Conclusion

Coding-related habits are deeply entrenched for experienced developers. In this work, we explore habit formation and behavior change literature to support replacing manual coding habits with agentic coding behaviors. We primarily rely on Fogg’s B=MAP framework, which proposes that for a behavior to occur, motivation, ability, and a timely prompt must align simultaneously. Internally, we developed a one-click Diagnose Error button to help users quickly fix bugs during notebook coding workflows. This button achieved high initial adoption, which may be explained by the design aligning with B=MAP’s three elements. However, habit formation requires a shift in mindset: agentic tools are not features to try once, but replacements for entrenched workflows. We propose four preliminary design principles to support a shift from one-off wins to repeated wins that form stable agentic coding habits: (1) leverage organic disruptions during users’ workflows, (2) design first experiences for plausible wins, (3) reduce friction with contextual prompting affordances, and (4) offer follow-up affordances.

Acknowledgments

The authors thank Databricks’s Assistant product, engineering, and design teams for their collaboration who pioneered a solution that helped users get acquainted with Agent Mode. Perplexity AI was used to conduct preliminary literature searches and identify

relevant research articles during the background research phase of this work. Claude (Anthropic) was used for editing assistance and brainstorming during the revision process, in accordance with the authors' AI use pledge. Databricks's Agent Mode was used to analyze internal telemetry data and generate statistical insights presented in this work, including the 75–77% adoption metrics and related analysis. All text was written by the authors, and all findings were subsequently verified and interpreted by the authors.

References

- [1] Shraddha Barke, Michael B James, and Nadia Polikarpova. 2023. Grounded copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111.
- [2] Aylin Caliskan, Fabian Yamaguchi, Edwin Dauber, Richard Harang, Konrad Rieck, Rachel Greenstadt, and Arvind Narayanan. 2015. When coding style survives compilation: De-anonymizing programmers from executable binaries. *arXiv preprint arXiv:1512.08546* (2015).
- [3] Aylin Caliskan-Islam, Richard Harang, Andrew Liu, Arvind Narayanan, Clare Voss, Fabian Yamaguchi, and Rachel Greenstadt. 2015. De-anonymizing programmers via code stylometry. In *24th USENIX Security Symposium (USENIX Security 15)*. USENIX Association, Washington, D.C., 255–270.
- [4] Fred D. Davis. 1989. Perceived Usefulness, Perceived Ease of Use, and User Acceptance of Information Technology. *MIS Quarterly* 13, 3 (1989), 319–340. doi:10.2307/249008
- [5] Brian J Fogg. 2009. A behavior model for persuasive design. In *Proceedings of the 4th International Conference on Persuasive Technology*. ACM, New York, NY, USA, 1–7.
- [6] Benjamin Gardner, Phillippa Lally, and Jane Wardle. 2012. Making health habitual: the psychology of 'habit-formation' and general practice. *The British Journal of General Practice* 62, 605 (2012), 664.
- [7] Daniel Kahneman. 2011. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, New York.
- [8] Phillippa Lally, Cornelia HM Van Jaarsveld, Henry WW Potts, and Jane Wardle. 2010. How are habits formed: Modelling habit formation in the real world. *European journal of social psychology* 40, 6 (2010), 998–1009.
- [9] Jenny T. Liang, Chenyang Yang, and Brad A. Myers. 2024. A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. ACM, New York, NY, USA, 616–628. doi:10.1145/3597503.3608128
- [10] Qianou Ma, Weirui Peng, Hua Shen, Kenneth Koedinger, and Tongshuang Wu. 2024. What You Say = What You Want? Teaching Humans to Articulate Requirements for LLMs. arXiv:2409.08775 [cs.HC]
- [11] Raymond G Miltenberger, R Wayne Fuqua, and Douglas W Woods. 1998. Applying behavior analysis to clinical problems: Review and analysis of habit reversal. *Journal of Applied Behavior Analysis* 31, 3 (1998), 447–469.
- [12] Everett M. Rogers. 2003. *Diffusion of Innovations* (5th ed.). Free Press, New York.
- [13] Nathan Rosenblum, Xiaojin Zhu, and Barton P Miller. 2011. Who wrote this code? Identifying the authors of program binaries. In *European Symposium on Research in Computer Security*. Springer, Berlin, Heidelberg, 172–189.
- [14] Nimisha Roy, Omojokun Olufisayo, and Oleksandr Horiello. 2025. Empowering Future Software Engineers: Integrating AI Tools into Advanced CS Curriculum. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 2*. ACM, New York, NY, USA, 1747–1747.
- [15] Daniel Russo. 2024. Navigating the Complexity of Generative AI Adoption in Software Engineering. *ACM Transactions on Software Engineering and Methodology* 33, 5, Article 135 (2024), 50 pages. doi:10.1145/3652154
- [16] Bas Verplanken, Ian Walker, Adrian Davis, and Michaela Jurasek. 2008. Context change and travel mode choice: Combining the habit discontinuity and self-activation hypotheses. *Journal of Environmental Psychology* 28, 2 (2008), 121–127.
- [17] Wendy Wood and David T Neal. 2007. A new look at habits and the habit-goal interface. *Psychological review* 114, 4 (2007), 843.